```
library(DBI)
library(bigrquery)

project_id <- "YOUR_PROJECT"
dataset_id <- "YOUR_DATASET"
billing_project <- "YOUR_BILLING_PROJECT"

con <- dbConnect(
  bigquery(),
  project = project_id,
  dataset = dataset_id,
  billing = billing_project
)



1-

sql_month_keys <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_month_keys_25m`

PARTITION BY feature_month
CLUSTER BY party_id

AS

SELECT
    s.party_id,

    DATE(s.observation_date) AS observation_date,

    DATE(s.first_qualifying_alert_date)
       AS first_qualifying_alert_date,

    CAST(s.label_3m AS INT64) AS label_3m,

    month_offset,

    LAST_DAY(
       DATE_SUB(
          DATE_TRUNC(
             DATE(s.observation_date),
             MONTH
          ),
          INTERVAL month_offset MONTH
       ),
       MONTH
```

```
    ) AS feature_month,

    CASE
      WHEN month_offset = 24 THEN 1
      ELSE 0
    END AS seed_month_flag

FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023` s

CROSS JOIN
  UNNEST(GENERATE_ARRAY(0, 24)) AS month_offset
"

DBI::dbExecute(con, sql_month_keys)
```

2-

```
sql_trades_pit <- "
CREATE OR REPLACE TABLE
 `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_trades_25m`

PARTITION BY feature_month
CLUSTER BY party_id, acct_id

AS

SELECT

  k.party_id,

  k.observation_date,

  k.first_qualifying_alert_date,

  k.label_3m,

  k.month_offset,

  k.feature_month,

  k.seed_month_flag,

  /* Account identifiers */

  t.trades_seq_num,
```

```
    t.acct_id,
    t.acct_num,
    t.acct_typ_cde,

    /* Account dates */

    t.open_dte,
    t.clsd_dte,
    t.paid_out_dte,
    t.eff_dte,

    /* Main behavioral fields */

    t.acct_mop_cde,

    t.curr_bal_amt,
    t.past_due_amt,
    t.cr_lmt_amt,

    /* Additional useful account-level information */

    t.high_cr_amt,
    t.payt_amt,
    t.payt_due_amt,
    t.payt_dte,

    t.max_delq_amt,
    t.max_delq_dte,
    t.max_delq_rte_cde,

    t.frst_delq_dte,
    t.frst_rpt_dte,

    /* Original source snapshot date */

    DATE(t.data_date) AS source_data_date

FROM
    `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_month_keys_25m` k

INNER JOIN
    `YOUR_PROJECT.YOUR_SOURCE_DATASET.trades` t

    ON k.party_id = t.party_id

   AND DATE(t.data_date) = k.feature_month
"
```

```
DBI::dbExecute(con, sql_trades_pit)
```

3-

```
sql_mop_transitions <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_mop_transitions_25m`

PARTITION BY feature_month
CLUSTER BY party_id, acct_id

AS

WITH mop_base AS (

    SELECT

        party_id,
        observation_date,
        first_qualifying_alert_date,
        label_3m,

        month_offset,
        feature_month,
        seed_month_flag,

        acct_id,
        acct_num,
        acct_typ_cde,

        acct_mop_cde,

        open_dte,
        clsd_dte,
        eff_dte,

        curr_bal_amt,
        past_due_amt,
        cr_lmt_amt,

        /* ====================================================
           MOP numeric DPD representation

           Only statuses with an explicitly supported
           delinquency ordering are mapped.
           ==================================================== */
```

```
      CASE
        WHEN acct_mop_cde = '01' THEN 0
        WHEN acct_mop_cde = '02' THEN 30
        WHEN acct_mop_cde = '03' THEN 60
        WHEN acct_mop_cde = '04' THEN 90
        WHEN acct_mop_cde = '05' THEN 120
        WHEN acct_mop_cde = '06' THEN 150
        ELSE NULL
      END AS mop_dpd,

      /* ====================================================
        Adverse-state indicator
        ==================================================== */

      CASE
        WHEN acct_mop_cde IN (
          '02',
          '03',
          '04',
          '05',
          '06',
          '8A',
          '8B',
          '8P',
          '09',
          '9B',
          '9P'
        )
        THEN 1
        ELSE 0
      END AS adverse_flag,

      CASE
        WHEN acct_mop_cde = 'UR'
        THEN 1
        ELSE 0
      END AS unrated_flag

    FROM
      `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_trades_25m`
),


mop_lag AS (

    SELECT
```

```
        *,

        LAG(acct_mop_cde) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_mop_cde,

        LAG(mop_dpd) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_mop_dpd,

        LAG(adverse_flag) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_adverse_flag,

        LAG(unrated_flag) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_unrated_flag,

        LAG(feature_month) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY feature_month
        ) AS previous_feature_month

    FROM mop_base
),


transitions AS (

    SELECT

        *,

        /* ====================================================
           Verify that previous snapshot is exactly prior month.

           If an account disappears for several months and
           reappears, we do NOT treat that as a direct
           month-to-month transition.
           ==================================================== */

        CASE
            WHEN previous_feature_month =
                LAST_DAY(
```

```
                DATE_SUB(feature_month, INTERVAL 1 MONTH),
                MONTH
            )
        THEN 1
        ELSE 0
    END AS consecutive_month_flag

  FROM mop_lag
)


SELECT

  party_id,
  observation_date,
  first_qualifying_alert_date,
  label_3m,

  month_offset,
  feature_month,
  seed_month_flag,

  acct_id,
  acct_num,
  acct_typ_cde,

  previous_feature_month,

  previous_mop_cde,
  acct_mop_cde AS current_mop_cde,

  previous_mop_dpd,
  mop_dpd AS current_mop_dpd,

  consecutive_month_flag,


  /* ============================================================
     Any MOP level change
     ============================================================ */

  CASE
     WHEN consecutive_month_flag = 1
      AND previous_mop_cde IS NOT NULL
      AND acct_mop_cde IS NOT NULL
      AND previous_mop_cde != acct_mop_cde
     THEN 1
     ELSE 0
```

```
END AS mop_changed_flag,


/* ========================================================
   Deterioration

   Only calculated where both MOP values have a supported
   ordinal DPD interpretation.
   ======================================================== */

CASE
    WHEN consecutive_month_flag = 1
     AND previous_mop_dpd IS NOT NULL
     AND mop_dpd IS NOT NULL
     AND mop_dpd > previous_mop_dpd
    THEN 1
    ELSE 0
END AS deterioration_flag,


/* ========================================================
   Improvement
   ======================================================== */

CASE
    WHEN consecutive_month_flag = 1
     AND previous_mop_dpd IS NOT NULL
     AND mop_dpd IS NOT NULL
     AND mop_dpd < previous_mop_dpd
    THEN 1
    ELSE 0
END AS improvement_flag,


/* ========================================================
   DPD jump

   Signed change.
   Positive = deterioration
   Negative = improvement
   ======================================================== */

CASE
    WHEN consecutive_month_flag = 1
     AND previous_mop_dpd IS NOT NULL
     AND mop_dpd IS NOT NULL
    THEN mop_dpd - previous_mop_dpd
    ELSE NULL
```

```
END AS dpd_jump,


/* ============================================================
   Entered adverse status
   ============================================================ */

CASE
    WHEN consecutive_month_flag = 1
     AND previous_adverse_flag = 0
     AND adverse_flag = 1
    THEN 1
    ELSE 0
END AS adverse_entry_flag,


/* ============================================================
   Resolved adverse status

   Conservative definition:
   adverse status -> current ('01')
   ============================================================ */

CASE
    WHEN consecutive_month_flag = 1
     AND previous_adverse_flag = 1
     AND acct_mop_cde = '01'
    THEN 1
    ELSE 0
END AS adverse_resolved_flag,


/* ============================================================
   Became UR
   ============================================================ */

CASE
    WHEN consecutive_month_flag = 1
     AND previous_unrated_flag = 0
     AND unrated_flag = 1
    THEN 1
    ELSE 0
END AS became_unrated_flag,


/* ============================================================
   Left UR
   ============================================================ */
```

```
    CASE
      WHEN consecutive_month_flag = 1
       AND previous_unrated_flag = 1
       AND unrated_flag = 0
      THEN 1
      ELSE 0
    END AS left_unrated_flag

FROM transitions
"

DBI::dbExecute(con, sql_mop_transitions)
```

4-

```
sql_transition_monthly <- "
CREATE OR REPLACE TABLE
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_transition_monthly`

PARTITION BY feature_month
CLUSTER BY party_id

AS

WITH transition_agg AS (

    SELECT

        party_id,
        observation_date,
        feature_month,
        month_offset,
        seed_month_flag,

        /* Number of accounts with any MOP change */
        SUM(mop_changed_flag)
            AS mop_changed_count,

        /* Deterioration / improvement */
        SUM(deterioration_flag)
            AS deterioration_count,

        SUM(improvement_flag)
            AS improvement_count,

        /* Adverse state transitions */
```

```
    SUM(adverse_entry_flag)
      AS adverse_entry_count,

    SUM(adverse_resolved_flag)
      AS adverse_resolved_count,

    /* UR transitions */
    SUM(became_unrated_flag)
      AS became_unrated_count,

    SUM(left_unrated_flag)
      AS left_unrated_count,

    /* Largest positive DPD deterioration observed
       during the month */
    MAX(
      CASE
        WHEN dpd_jump > 0 THEN dpd_jump
        ELSE 0
      END
    ) AS max_dpd_jump

  FROM
    `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_mop_transitions_25m`

  GROUP BY
    party_id,
    observation_date,
    feature_month,
    month_offset,
    seed_month_flag
)


SELECT

  /* ============================================================
     Use month_keys as backbone.

     This is important because a party-month must continue
     existing even if no account had a MOP transition.
     ============================================================ */

  k.party_id,

  k.observation_date,

  k.first_qualifying_alert_date,
```

```
    k.label_3m,

    k.month_offset,

    k.feature_month,

    k.seed_month_flag,


    /* ============================================================
       Monthly transition measures

       If no transition rows exist for the party-month,
       event counts are legitimately zero.
       ============================================================ */

    COALESCE(t.mop_changed_count, 0)
       AS mop_changed_count,

    COALESCE(t.deterioration_count, 0)
       AS deterioration_count,

    COALESCE(t.improvement_count, 0)
       AS improvement_count,

    COALESCE(t.adverse_entry_count, 0)
       AS adverse_entry_count,

    COALESCE(t.adverse_resolved_count, 0)
       AS adverse_resolved_count,

    COALESCE(t.became_unrated_count, 0)
       AS became_unrated_count,

    COALESCE(t.left_unrated_count, 0)
       AS left_unrated_count,

    COALESCE(t.max_dpd_jump, 0)
       AS max_dpd_jump


FROM
  `YOUR_PROJECT.YOUR_DATASET.design_a_500k_2023_month_keys_25m` k

LEFT JOIN transition_agg t

    ON k.party_id = t.party_id
```

```
  AND k.observation_date = t.observation_date
  AND k.feature_month = t.feature_month
"

DBI::dbExecute(con, sql_transition_monthly)
```